Class: Algorithms & Data Structures Semester: Spring 2025

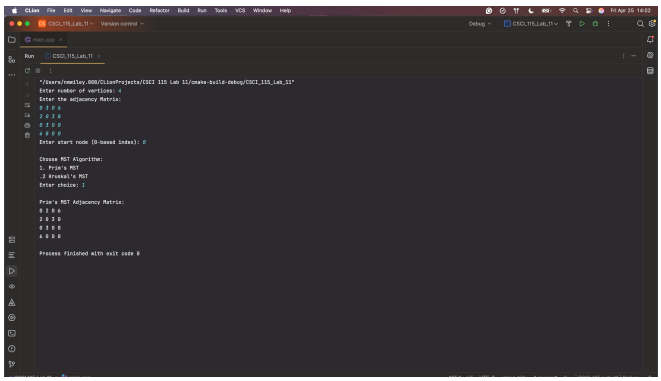
Noah Wiley
noahwiley@mail.fresnostate.edu
Lab Number - Section 09

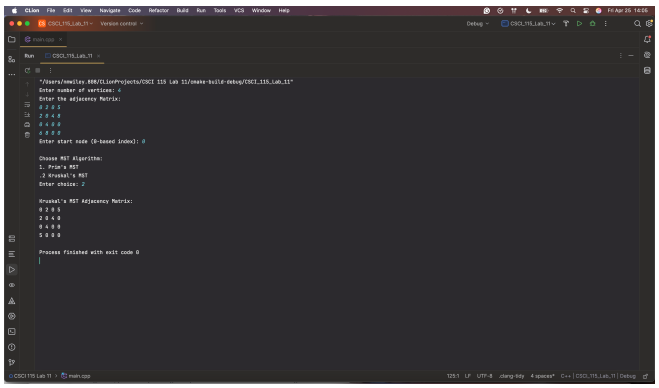
CSCI 115 Lab 11; Minimum Spanning Tree - Prim's & Kruskal's Algorithm

- Statement Of Objectives
  - The objective of this lab is to implement and analyze two minimum spanning tree (MST) algorithms. Prim's algorithms: Prim' s algorithm and Kruskal's algorithm. We aim to understand the operational differences between these two algorithms, implement them using C++ 20 in CLion, and evaluate their outputs on a given graph. The significance of this lab lies in learning efficient methods to connect all vertices in a weighted undirected graph with the minimal possible total edge weight. Major accomplishments include correctly implementing both MST algorithms, validating their outputs, and analyzing their similarities. This report details the experimental procedure, analysis, encountered challenges, and conclusions drawn from this experiment.

- Experimental Procedure
  - Implemented the utility function printMatrix to display adjacency matrices
  - Implemented Prim's algorithm in primsMST, accepting an adjacency matrix and a start node
  - Implemented Kruskal's algorithm in kruskalMST, including helper functions find and unionSets for unionSets for union-find operations
  - Built and ran the program in CLion with C++20 settings
  - Captured results and compared the MST outputs between the two algorithms.

- Analysis





- The MST generated by both Prim's and Kruskal's algorithms contians three edges: (0-1, weight 2), (1-2, weight 3), and (0-3, weight 6). The total minimum spanning tree cost calculated from these edges is; $2 + 3 + 6 = 11$
- Both algorithms produced the exact same MST structure, confirming that they were correctly implemented .
- Prim's algorithm builds the MST by progressively adding the nearest vertex to the growing MST set, starting from the user-specified node (node 0 in this case). It prioritized adding the smallest edges (2, 3and 6) while avoiding cycles. On the other hand, Kruskal's algorithm treated all edges globally, sorted them by weight, and added them one by one while preventing cycles using the union-find data structure.
- Even though the two algorithms use different strategies (Prim's is greedy expansion from a node, Kruskal's is global edge consideration), they ultimately lead to the same MST because the minimum spanning tree is unique when all edge weights are distinct or structured cleanly.
- This shows the consistency and correctness of both implementations. Moreover, both outputs match theoretical expectations for MST construction in undirected, weighted graphs.

- Encountered Problems
  - Initially, a linker error occurred stating "Undefined symbols for architecture arm 64" related to the printMatrix function. The issue was caused by a missing or misplaced function definition. This was resolved by ensuring that printMatrix was defined above its first use in the file. Another minor adjustment was required for setting the C++ standard to C++ 20 explicitly in the CLion project settings to avoid any compilation mismatches.

- Conclusions
  - Successfully implemented Prim's and Kruskal's MST algorithms in C++20
  - Understood the operational differences and similarities between Prim's and Kruskal's approaches
  - Observed that both algorithms correctly build the MST for the sample graph
  - Gained practical experience in handling function dependencies and linker errors

- References
  - Lab Lecture Slide on Minimum Spanning Trees